Prompt Writing Guideline

HarmonyGames · Fixed environment date: February 14, 2026, 00:00 UTC · Updated September 26, 2026

How to write a prompt that is hard because the universe is hard — and that reads like a colleague typed it, because under Spec V2 you have to type it yourself.

Change log

Date	Updates
Sep 26, 2026	Rewritten for Spec V2. **LLM-written prompts prohibited**; AI-text detection section added. Baseline model updated to Opus 5.5. Hinting and golden-solution references removed. Tool-call minimum rule updated. Super-Long-Horizon marked paused. Examples rebuilt on the ML-selected task.
Aug 2026	Original release.

01 · What a task looks like

A task is a short, informal message from one member of staff to an assistant, dropped into a universe that has been prepared so the message is hard to act on.

```
[something is visibly wrong, said the way a colleague would say it]

[work out what actually happened]          <- investigation

[then do something about it]               <- action, across 2+ services

[a constraint, a deadline, or who to tell]
```

> **THE ONE RULE THAT GOVERNS EVERYTHING ELSE**
>
> Difficulty has to come from the **environment**, never from a vague prompt. A task that is hard because the information is scattered and some of it is stale is exactly right. A task that is hard because nobody can tell what was asked fails Unique Ground Truth and is rejected.

02 · Your assignment

Five axes are fixed for you. Your scenario has to fit all five.

Axis	What it fixes	Mismatch severity
Business function	The part of the company the work belongs to	**Non-fail** if ambiguous between yours and another
Persona	Who is asking, and what the agent may read	**Fail** if the prompt could not plausibly come from that person
Competency	The type of work	**Non-fail** if ambiguous between yours and another
Task skill	What the agent must be good at	**Non-fail** if only marginally exercised
Task type	How large the task must be	**Fail** if the real tool-call count lands under your tier

2.1 Assigned persona — follow it exactly

You may not switch personas, and you may not build around a persona's limitations by pretending they do not apply. The persona sets the voice, the remit and the read scope. The **Persona ACL Policy** guideline covers the read scope in full.

2.2 Assigned competency — the work, not the job title

The competency drives what kind of work the task is. It does not have to be the obvious match for your persona. A Game Engineer assigned the Finance competency writes a budgeting or forecasting task, because a Game Engineer can plausibly be pulled into one. Mentioning a budget figure inside an engineering task does not make it a Finance task.

2.3 Assigned task skill — one is mandatory, stacking is optional

The assigned skill has to be genuinely exercised, not nodded at. Stacking a second is encouraged and is how most strong tasks end up structured — the ML-selected example stacks Analysis and Synthesis with Retrieval and Actions.

2.4 Assigned task type — how big the task is

Tier	Tool calls	Share of the project
Normal	50 – 100	50%
Long-Horizon	100 – 500+	50%
Super-Long-Horizon	500+	**Paused** — still visible on the task screen, but not assigned

HOW THE MINIMUM IS COUNTED *[CHANGED 09/26]*

If any of your six rollouts scored **100% on your rubric**, that rollout is a **golden run** and the tool-call minimum applies to it alone. If none did, the minimum applies to the **average across all six**. A golden run is a run the model produced, not a document you write — the Oracle Events set does not enter this calculation.

Padding does not help. Blocked calls and repeated calls add nothing to real depth, and an inflated count is a failing error in its own right.

03 · Inject the context into the universe

Rule zero — seed the context, don't type it in

✗ "Hi, can you help me do…" followed by every requirement spelled out, exact ticket IDs, exact document titles, and numbered steps.

✓ Put the detail in a Slack channel, then say "check #channel and follow what's there". Name the problem, not the method. "that revenue thing", "the bug people keep complaining about".

Everything about how the universe should be prepared — the six consistency rules, the clock, what makes seeded data read as real — is in the **Universe and Tools** guideline. Two points bear repeating here because they are prompt-side failures, not universe-side ones:

- **Seed the symptom, never the conclusion.** A prompt that states the root cause is **pre-solved**, which is a failing error. There is nothing left to diagnose.
- **Every rubric has to trace back to the prompt.** If you seeded a requirement into the universe and the prompt gives the agent no reason to go looking for it, the resulting criterion is unauthorized.

04 · Writing the prompt

4.1 Write it yourself V2

LLM USE IS PROHIBITED

Prompts must be written by you, from scratch. Using an LLM to draft the prompt is prohibited, and so is using one to **polish** it. The eval sandbox may be used to query the universe; it may not be used to write your prompt. An **AI-text detector runs on your prompt** before anything else in the pipeline.

An LLM-sounding prompt is the single most common reason a prompt is rejected. Think about how you actually type when you are messaging an assistant in your own life. You do not capitalise every sentence. You do not proofread. You type what you need and hit enter.

Red flag	What to do instead
Perfectly capitalised and punctuated throughout	Mix it up. Skip a capital. Leave a typo in
Neatly enumerated steps 1, 2, 3	Let sentences run on and double back — "oh and also"

Red flag	What to do instead
Markdown — headings, bold, bullets, horizontal rules	Plain prose with no structure at all
Formal phrasing: “Please ensure”, “Kindly proceed to”	“can you check on the launch stuff”
Unnaturally complete — nothing left unsaid	Be vague exactly where a real person would be vague
Perfect ticket IDs and document titles	“whatever julia was talking about”

> **EXACT TITLES AND VALUES**
>
> Referencing an exact title or value is fine when it is already human-readable — `GEOGAMES_SPEC.docx` or `*ea96`. It is not fine when it is not, like `geogames_spec_23mf2ldf02lfasdlov23rmf.docx` or a full `agent_id` lifted from a run. A real person would never type the second kind.

4.2 The hard rules

Always	Never
Require real tool use — if general knowledge answers it, it is not a task	Name a tool, a parameter, or an MCP server
Span at least two services, ideally three	Pre-solve it by stating the root cause
Sound like a real person typing: first person, informal	Include internal IDs a real person would not know
State dates absolutely, anchored to February 14, 2026	Announce an injected failure or an async wait
Ask for investigation *and* action	Format it like a document
Keep every sentence load-bearing	Bolt on an unrelated request

On naming tools

Three severities, and they are not the same:

What you wrote	Severity	Why
“run `linear_create_issue` for each open item”	**Fail**	An actual MCP tool name
“check it via the Gmail MCP server”	**Non-fail**	Names the server, not the tool
“use the QuickBooks tool to pull the invoice”	**Non-fail**	Refers to a system as “the X tool”
“have a look in the finance sheet and the email thread”	**Clean**	Names services the way a colleague would

On bolt-ons

The test is simple: **if you can delete a sentence without affecting anything else in the prompt, it should not be there.** An unrelated extra request does not add horizon, it adds a second task.

4.3 Implicit requirements

Implicit requirements are encouraged — things a competent assistant would obviously infer, like checking the data before filing a bug, or telling the person who asked. Keep them genuinely inferable rather than hidden gotchas. A requirement nobody could reasonably infer produces an unauthorized rubric criterion, which is a scored defect.

05 · Difficulty characteristics to build in

Characteristic	Definition	How you build it
Ambiguity	The prompt under-specifies by design; the agent resolves the gaps from the environment	Seeding
Dynamic Change	What the agent learns early is overturned by what it finds later	Seeding
Multi-Source	Information split across three or more sources, with distractors and disagreements	Seeding
Async Tool Calls	Submit-then-wait tools; the agent parks the dependency and resumes	Configuration
Error Recovery	Forced tool failures; the agent retries, works around, or escalates	Configuration
Access Control	The agent must work within what its persona may read	Persona + seeding

All six are optional and all six stack. A single task can carry one or all of them.

The three you build by seeding

Ambiguity

Identify the details the agent needs — dates, names, priorities, constraints — remove them from the prompt, and plant them across Slack threads, email chains, doc comments, tickets and spreadsheet notes. Then add noise: related-but-irrelevant material nearby, so the agent has to separate signal from it.

✗ **Over-specified.** “The Q1 board deck on Drive has a revenue figure that disagrees with the finance sheet in row 47. Fix the deck and email Frederick.”

✓ **Ambiguous.** “Frederick mentioned the board deck numbers looked off — can you look into it?” with the real discrepancy sitting between the Drive deck and the Sheets data.

Dynamic Change

Create an initial state the agent discovers first, then plant a contradicting update it reaches later. **Discovery order is what makes it land.** Put the correction where a plausible investigation path meets it *second*, after the thing it

overturns.

Multi-Source

Spread what is needed across at least three systems, add lookalike distractors — old threads, archived tickets, draft docs — and introduce a disagreement so the agent has to judge which source is authoritative.

The two you configure

Setting	Behaviour	What it tests
Error rate **1.0**	Fails on every call. A retry will never succeed — a permanent outage	Whether the agent finds a workaround or escalates
Error rate **below 1.0**	Fails that proportion of calls, and is marked transient	Whether the agent retries
Async wait	Search becomes submit → check → get, with a 30–35 second delay per call	Whether the agent parks the dependency and resumes

THREE HARD RULES ON INJECTED FRICTION

Your task must still be solvable without any tool you kill. Name the alternative path out loud before you configure anything, confirm it exists in the universe, and show it in your Oracle Events.

Async affects search only, never fetch. If your prompt names the exact document, channel or ticket, the agent fetches it directly and never pays the delay. Write the prompt so discovery is unavoidable.

A forced error on a scoped tool is indistinguishable from an ACL denial. If your task depends on the agent reasoning about why it cannot see something, do not also force an error on that tool.

06 · How agents search

- Search is fuzzy and returns ranked candidates. Fetch by ID is exact and instant.
- Naming an exact artifact in the prompt collapses discovery to a single fetch. That is usually the opposite of what you want.
- Agents stop early. A message that *sounds* like a conclusion will end the investigation, even when the thread continues afterwards. That is a legitimate and effective trap — the ML-selected example is built on one.
- Agents will bypass a slow path if a faster one exists. If you configured an async wait, open the trajectory and confirm the submit → check → get sequence actually appears.

07 · The ambiguity line

✓ **Discovery ambiguity.** The agent does not know where the answer is, which sources matter, or which of two sources is authoritative — but every reasonable run still ends in the same final state.

✗ **Decision ambiguity.** Two readings lead to two different write actions. Act now or defer. Escalate to A or to B. File in X or in Y.

UNIQUE GROUND TRUTH HAS NO MIDDLE BAND

If two careful runs land the agent in two different final universe states, the prompt fails — even if most people would pick the same reading. This dimension scores 1–2 or 5, with nothing in between.

08 · Before you submit the prompt

- You wrote every word of it yourself, without an LLM drafting or polishing.
- It reads like fast, unproofread typing rather than a document.
- It names no tool, no MCP server and no parameter.
- It states no root cause the agent is supposed to find.
- Every sentence is load-bearing — nothing can be deleted without consequence.
- Every date is absolute and sits before February 14, 2026.
- Every name, ID and figure has been checked against the explorer, not against memory.
- It could plausibly have been sent by your assigned persona, about work in your assigned competency.
- It requires at least two services, ideally three, and both investigation and action.
- Two careful runs would end in the same final state.
- `/prompt` returned PASS or NON-FAIL. A FAIL there is fatal — with hinting removed, nothing downstream rescues an unsolvable prompt.
